

F1R3Score

Building, deploying and using `f1r3score`, the player for the score calculus

F1R3FLY.io

Documentation — 27 September 2026

Against `F1R3FLY-io/F1R3Score` commit `7416bc0`, which implements revision 2 of *The f1r3score Player* specification

Abstract

`f1r3score` is a Rust command-line player and library for the score calculus of *Scores as Processes*. A score is a process in a variant of the rho calculus, and a performance is one run of it. The player reads a textual score, elaborates it to a closed core term, and executes a run. The run can be written out as a table, JSON, a replayable trace of records, a Standard MIDI File, or live MIDI.

This guide covers three things. It shows how to build the workspace and run its conformance suite. It covers deploying the player as a binary, as a library, and as a reproducible archive of performances. And it shows how to use the player and write scores for it. Every command and every output quoted here was run against the commit named above, and the guide says where it could not verify something. Section 9 lists where the implementation departs from the specification or from its own documentation.

Contents

1	Introduction	3
1.1	What <code>f1r3score</code> is	3
1.2	Sources	3
1.3	How this guide was checked	3
2	Concepts in brief	4
3	Building	5
3.1	Prerequisites	5
3.2	Getting the source and building	5
3.3	Cargo features	5
3.4	Build profiles	5
3.5	Running the tests	5
3.6	The cross-implementation differential	6
3.7	Building for WebAssembly	6
3.8	Build problems	6
4	Deploying	7
4.1	Installing the binary	7
4.2	Pin the dependency graph	7
4.3	Reproducibility and archiving performances	7
4.4	Embedding the player as a library	7

4.5	Live MIDI	8
4.6	Continuous integration (proposed)	8
4.7	A container image (proposed)	9
5	Using the player	9
5.1	Quick start	9
5.2	Commands	10
5.3	Options for <code>play</code>	10
5.4	Chance sources	11
5.5	Stream labels	11
5.6	Schedulers, modes and collection	12
5.7	Human choice	12
5.8	Outputs	12
5.9	Exit codes and stop reasons	13
6	Writing scores	13
6.1	File structure	13
6.2	Alphabets	14
6.3	Processes	15
6.4	Definitions	15
6.5	Clauses	16
6.6	The checkable fragment	17
6.7	The standard library	17
7	Worked examples	18
7.1	A voice with a style	18
7.2	The shipped examples	18
7.3	Transcendental numbers as players	19
7.4	Importing a drawn machine	19
8	Troubleshooting	19
9	Known limitations and discrepancies	20
A	Repository layout	21
B	Conformance tests	22

1 Introduction

1.1 What `f1r3score` is

The score calculus proposes a musical notation whose primary objects are exchange and preference rather than a single fixed performance: who listens to whom, which continuation a style favours, what lies easily under a player's hands. `f1r3score` makes that notation playable. It has three layers:

- a *surface syntax*, a dialect of the `f1r3|@ng` control-flow sublanguage. It has decorated names, alphabet declarations and an elaboration-time macro layer, with files ending in `.score`;
- a *standard library*, `std.score`, written entirely in that syntax. It defines written notes, reflective voices, hockets, call and response, keys, keyboards and chimeric instruments;
- an *engine* that knows nothing of voices, keys or idioms. It matches receipts and messages, weighs the candidates with graded where-clauses, resolves contention by maximal matchings, keeps the record of every communication, and reads notes off records.

1.2 Sources

The guide draws on three sources in the `F1R3FLY-io` GitHub organisation:

- `F1R3Score`, the implementation. It contains `README.md`, `DESIGN.md` (decisions, deviations and verification status), the examples, and a copy of the note in `docs/`;
- `publications/f1r3score/score-calculus.tex`, the note *Scores as Processes*, which defines the calculus;
- `publications/f1r3score/f1r3score-player-spec.tex`, the player specification (revision 2), together with `skeinsim.py`, the reference simulator.

Section numbers of the form “§5.2 of the note” refer to *Scores as Processes*. Requirement and decision numbers refer to the specification.

1.3 How this guide was checked

The guide was written against commit `7416bc0` on Ubuntu 24.04 (x86_64), with Rust 1.75.0 from the distribution packages. That is also the workspace's declared minimum, `rust-version = "1.75"`. On that machine:

- the release build succeeds with no warnings, and so does the build with the `live` feature;
- `cargo test --release` passes all 70 tests;
- the cross-implementation differential against `skeinsim.py` agrees on every scenario (run with 10 random scores);
- every command in Sections 5 and 7 was run, and the outputs shown are the actual outputs.

Two things could not be verified: the `wasm32` build of the core crates, and live MIDI on a real device (§9).

2 Concepts in brief

This section gives the minimum of the calculus needed to read the rest of the guide. The note is the authority.

Names carry notes. A name is a triple $\langle @P, \tau, d \rangle$: a quoted process $@P$ (its *location*), a timbre τ , and a datum d . The datum is either a pitch or a duration, and either decoration component may be left open, written $_$. A name whose datum is a pitch is a *channel*; a name whose datum is a duration is a *co-channel*. The *general name* $@P$ is $\langle @P, _, _ \rangle$.

Every communication is a note. A receipt for $(y \leftarrow x \text{ where } \text{psi}) \{ P \}$ and a message $x' ! (Q, \text{tau}, d)$ communicate when their subjects *match*. That means three things hold: their locations are equivalent, their timbres meet in a concrete timbre, and their data are concrete and of opposite polarity (one a pitch, the other a duration). Each firing yields a *record*, and the *playback* function ν reads a note off it: the timbre, the pitch, and the duration. There are no silent steps.

Null notes. The rest r and the null duration eps (length 0) let the calculus do bookkeeping, such as a server fetching its own code, as communications that are notes of zero length. These *null notes* are recorded but excluded from the performance.

Clauses weigh choices. A receipt's where-clause evaluates to an exact non-negative rational on each candidate. A candidate is *available* when its weight is positive. Available candidates that share a receipt or a message form a *contention set*. A set is resolved by choosing a maximal matching with probability proportional to the product of its weights, and firing all of it. Clauses multiply, so style, register and the physicality of an instrument can be separate factors whose product is the voice's law.

Time. Every top-level process carries a stamp. A communication's onset is the later of its two stamps, and its continuation is stamped at the onset plus the note's length. Sets are resolved in onset order.

Voices are reflective. There is no recursion former. An unbounded voice is a server that keeps its own code on a code channel and re-sends it each time it is read (§5.2 of the note). Definitions in the surface syntax are therefore macros with an acyclic call graph.

Musicians and instruments. A *key* is a receipt holding a pitch, and it belongs to the instrument. A *touch* is a message holding a duration, and it belongs to the player. A keyboard installs keys at key locations. A player sends touches, and $;$ makes a player wait until its note has ended (Definition 9.3 of the note). A *chimera* holds a key per timbre at one location, so a touch that leaves its timbre open is answered in whichever timbre the keys' clauses favour.

3 Building

3.1 Prerequisites

need	for
Rust $\geq$ 1.75 and Cargo	everything. Any of rustup, the distribution packages (Ubuntu 24.04: <code>apt install rustc cargo</code> gives 1.75.0), or Homebrew will do.
network access to crates.io	the first build only. The default build depends only on <code>num-bigint</code> , <code>num-rational</code> , <code>num-traits</code> and <code>num-integer</code> .
ALSA headers and <code>pkg-config</code> (Linux)	the optional <code>live</code> feature only: <code>apt install libasound2-dev pkg-config</code> . On macOS <code>midir</code> uses CoreMIDI, and on Windows WinMM; neither needs extra packages.
Python 3 and a checkout of publications a <code>wasm32-unknown-unknown</code> standard library	the cross-implementation differential only (§3.6). building the core crates for the browser (§3.7).

3.2 Getting the source and building

```
git clone https://github.com/F1R3FLY-io/F1R3Score.git
cd F1R3Score
cargo build --release
./target/release/flr3score --help
```

A cold release build takes about a minute. The binary is `target/release/flr3score`. It is self-contained: `std.score` is compiled in with `include_str!`, so no data files need to accompany it.

3.3 Cargo features

feature	crate	default	effect
<code>spigot</code>	<code>score-chance</code>	on	exact digit streams of π , e , $\ln 2$, Liouville's constant, Champernowne's constant and the Thue–Morse sequence, in any base: the <code>spigot: chance sources</code>
<code>live</code>	<code>flr3score</code> , <code>score-render</code>	off	live MIDI output through <code>midir</code> : the <code>--live</code> <code>PORT</code> option and the <code>ports</code> subcommand

```
cargo build --release --features live -p flr3score
./target/release/flr3score ports      # lists MIDI output ports
```

3.4 Build profiles

The workspace sets `opt-level = 1` for dev and `opt-level = 2` for test, because the conformance tests draw tens of thousands of notes. Use `--release` for anything musical. A 16-pitch $\times$ 5-duration voice plays at roughly 0.2–0.5 ms per note in a release build.

3.5 Running the tests

```
cargo test --release
```

This runs 70 tests and takes about two minutes. Nearly all of that is `tests/law.rs` (about 60 s) and `tests/suite.rs` (about 50 s) in `score-conformance`. What they cover:

test file	what it checks
score-core unit, congruence.rs	normal forms and matching; the congruence and round-trip property tests over wildcards (10^4 random pairs against an independent decision procedure)
score-syntax/tests/syntax.rs score-chance unit	parsing, elaboration, printing and re-parsing, and diagnostics PRNG determinism, spigot digits against known expansions, interval decoding
suite.rs, law.rs, t12.rs	the ported skeinsim suite T0–T12 with the note’s thresholds
instruments.rs	T13 (4-hands piano), T14 (chimera), and the instrument propositions
outputs.rs	records and playback, render against play, the MIDI golden files, replay, and detection of tampered choices
examples.rs	every file in examples/ elaborates and plays
portability.rs	the five core crates contain no I/O

To run a single file: `cargo test --release -p score-conformance --test instruments`.

3.6 The cross-implementation differential

The differential checks that the Rust player and the Python reference simulator emit the same records at every resolution, null notes included. The player’s traces drive `skeinsim.py`. It needs a checkout of `publications` and runs from the workspace root:

```
git clone https://github.com/F1R3FLY-io/publications.git ../publications
sh crates/score-conformance/run_differential.sh ../publications/flr3score
40
```

The second argument is the number of random small scores (default 40). The script ends with `differential: every scenario agrees, or exits 1 naming each scenario that differs`. It is not part of `cargo test`, because it needs Python and the other repository.

3.7 Building for WebAssembly

The five core crates (`score-core`, `score-logic`, `score-syntax`, `score-chance`, `score-engine`) perform no I/O. That property is enforced by `portability.rs`, and it is what makes them candidates for `wasm32-unknown-unknown`:

```
rustup target add wasm32-unknown-unknown
for c in score-core score-logic score-syntax score-chance score-engine;
do
  cargo build --release --target wasm32-unknown-unknown -p $c
done
```

Not verified.

Neither `DESIGN.md` nor this guide has seen that build succeed, because neither build machine had a `wasm32` standard library. Ubuntu does not package one for its `rustc`, so use `rustup`.

3.8 Build problems

error: package ‘...’ cannot be built because it requires rustc 1.75
The toolchain is too old; upgrade it.

```
failed to run custom build command for `alsa-sys`
  --features live on Linux without ALSA headers. Install libasound2-dev and
  pkg-config.

could not compile `num-traits` ... (wasm32)
  No standard library for that target is installed (§3.7).
```

4 Deploying

4.1 Installing the binary

```
cargo install --path crates/flr3score # into ~/.cargo/bin
cargo install --path crates/flr3score --features live # with live MIDI
```

On Linux the default binary links only `libc`, `libm` and `libgcc_s`, so it copies between glibc systems of the same or later vintage. It is about 15 MB unstripped; `strip` reduces that considerably. A fully static build (`--target x86_64-unknown-linux-musl`) should work for the default feature set, since there are no C dependencies, but it was not tried. The `live` build links ALSA dynamically and needs `libasound2` at run time.

4.2 Pin the dependency graph

`Cargo.lock` is not committed. For a binary that promises bit-for-bit reproducible runs it should be, so that every build resolves the same `num-*` crates. The versions resolved for this guide were `num-bigint 0.4.8`, `num-rational 0.4.2`, `num-traits 0.2.19`, `num-integer 0.1.47` and, with `live`, `midir 0.9.1` and `alsa 0.7.1`. Until the lock file is committed, build with the lock file of a known-good checkout, or with `cargo build --locked` once it is.

4.3 Reproducibility and archiving performances

A run is determined by four things: the score, the chance sources, the scheduler, and the matching and collection modes. Every trace records all of them in its header, together with the score's SHA-256 structural digest and the player version. Two properties follow for archiving:

- **Replay.** `flr3score replay TRACE --score SCORE` re-runs the score from the trace's choices. It refuses a score whose digest differs from the trace's, and exits 4 if any choice emits different notes. Archive the score together with its trace.
- **Playback.** `flr3score render TRACE` needs neither the score nor the engine. The trace's records contain everything playback reads, so a trace on its own is a durable performance.

Digests are structural. Reformatting a score, renaming bound variables, or changing stream labels (§5.5) leaves the digest unchanged; changing its meaning does not.

4.4 Embedding the player as a library

The crates can be used directly from another Rust program, for example the F1R3Games back end or a browser front end over the core crates. This program was compiled and run against the workspace:

```
// Cargo.toml: score-syntax, score-engine, score-chance, score-render
//           as path or git dependencies on the F1R3Score workspace
use score_engine::{Config, Engine, Limits, Sources};
use score_syntax::{load, NoFiles};
```

```

fn main() -> Result<(), String> {
    let src = std::fs::read_to_string("examples/hocket.score").map_err(|e|
    e.to_string())?;
    // NoFiles allows only "import std"; implement score_syntax::Loader
    // to resolve import "file.score" from wherever your scores live.
    let s = load("hocket.score", &src, &mut NoFiles).map_err(|d| d.to_string())?;
    let sources = Sources::new(score_chance::source("prng:42", 64)?);
    let mut e = Engine::new(s.arena, s.clauses, s.alph, &s.initial, s.window,
                           s.behavioural, Config::default(), sources)
        .map_err(|e| e.to_string())?;
    let limits = Limits { notes: Some(8), ..Default::default() };
    let mut notes = Vec::new();
    e.run(&limits, &mut |ev, _| notes.extend(ev.notes.iter().cloned()))
        .map_err(|e| e.to_string())?;
    let perf = score_render::performance_of(&notes, &e.alph); // drops null notes
    print!("{}", score_render::performance_table(&perf, &e.alph));
    let midi = score_render::midi::write(&perf, &e.alph, 120);
    std::fs::write("hocket.mid", &midi.bytes).map_err(|e| e.to_string())
}

```

The pieces an embedder can use:

- `Sources::bind(label, source)` binds stream labels, as `--stream` does.
- `Engine::step` advances one resolution at a time, for callers that interleave playing with something else.
- A type implementing `score_chance::Chance` can supply choices directly. A graphical chooser would be one; the CLI's `Human` type is the model to follow.

The engine's public types are those of revision 2 of the specification. They are not yet a stable API: the workspace version is 0.1.0.

4.5 Live MIDI

Build with `--features live`, list the available ports, and pass a substring of the port name:

```

flr3score ports
flr3score play examples/chimera.score --live "FluidSynth" --bpm 96

```

On a Linux host without a hardware synthesiser, a software synth such as FluidSynth or TiMidity++ provides an ALSA sequencer port. Containers and headless CI machines usually lack `/dev/snd/seq`; there ports fails with MIDI support could not be initialized (exit 3). Prefer `--midi` there.

Not verified.

Live MIDI compiles and links, but it has not been tested on a MIDI device. Section 9 also describes how its timing departs from the specification.

4.6 Continuous integration (proposed)

`portability.rs` and `DESIGN.md` both refer to a CI job at `.github/workflows/ci.yml`, but that file is not in the repository. The following workflow would cover what the design notes expect of CI: the tests, the wasm build and the differential. It has not been run.


```
# .github/workflows/ci.yml -- proposed, untested
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@1.75
      - run: sudo apt-get install -y libasound2-dev pkg-config
      - run: cargo build --release --features live
      - run: cargo test --release
  wasm:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
        with: { targets: wasm32-unknown-unknown }
      - run: |
          for c in score-core score-logic score-syntax score-chance score
          -engine; do
            cargo build --release --target wasm32-unknown-unknown -p $c
          done
  differential:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/checkout@v4
        with: { repository: F1R3FLY-io/publications, path: publications }
      - uses: dtolnay/rust-toolchain@1.75
      - run: sh crates/score-conformance/run_differential.sh publications
        /f1r3score 40
```

If publications is private, the second checkout needs a token with read access to it.

4.7 A container image (proposed)

For batch rendering, such as a service that turns scores or traces into MIDI files, a two-stage image keeps the runtime small. This Dockerfile is a template and has not been built:

```
FROM rust:1.75-slim AS build
WORKDIR /src
COPY . .
RUN cargo build --release -p f1r3score

FROM debian:bookworm-slim
COPY --from=build /src/target/release/f1r3score /usr/local/bin/f1r3score
WORKDIR /work
ENTRYPOINT ["f1r3score"]
# docker run --rm -v "$PWD:/work" f1r3score play piece.score --notes 64
--midi out.mid
```

5 Using the player

5.1 Quick start

```

B=target/release/flr3score
$B check examples/twinkle.score
$B play examples/twinkle.score --midi twinkl.e.mid
$B play examples/country_voice.score --notes 32 \
    --stream A=prng:3 --stream B=prng:4 --gc freshness \
    --trace run.jsonl --midi run.mid
$B replay run.jsonl --score examples/country_voice.score
$B render run.jsonl --midi again.mid

```

These commands, in order, check Twinkle, play it to a MIDI file, play 32 notes of two voices while writing a trace and MIDI, replay the trace against the score, and render the trace again without the score. The last `again.mid` is byte-identical to `run.mid`.

5.2 Commands

command	purpose
check SCORE	Parse, elaborate and lint. Prints the digest, the regime (<code>local</code> , or <code>window-behavioural</code> (depth <i>d</i>); see §6.6), and the number of terms and clauses.
expand SCORE	Print the elaborated core term, with every definition expanded. The output re-parses.
play SCORE [opts]	Run the score once (§5.3).
replay TRACE --score SCORE	Re-run from a trace's recorded choices and check that each emits the recorded notes. Accepts <code>--midi</code> , <code>--bpm</code> and <code>--format</code> ; the scheduler, mode and collection come from the trace.
render TRACE [opts]	Playback only: read the notes off the trace's records. Accepts <code>--midi</code> , <code>--bpm</code> , <code>--out</code> , <code>--format</code> and <code>--include-null</code> .
machine import FILE	Convert a machine exported by the instrument (indexed states) into a factor line over the score's alphabet.
--kind pitch dur [--dual]	
--score SCORE [--name N]	
ports	live builds only: list MIDI output ports.

check on Twinkle prints:

```

ok examples/twinkle.score
digest d5b61ae1d112ed6322f0981a125f2d82ce54cfea3277e89636bda081845af0a9
regime local
terms 163 clauses 1

```

5.3 Options for play

option	meaning (default)
--chance SPEC	default chance source for every stream not bound otherwise (<code>prng:0</code>)
--stream LABEL=SPEC	bind a stream label, a timbre, or <code>LABEL.pitch/LABEL.dur</code> to a source; repeatable
--scheduler S	order of simultaneously due sets: <code>canonical</code> , <code>random:SEED</code> , or <code>by-timbre:A,B,...</code> (<code>canonical</code>)
--mode max one	fire a maximal matching, or a single candidate as a diagnostic (<code>max</code>)
--gc off freshness	collect inert record locations (<code>off</code>)
--notes N	stop after N notes of positive length
--until T	stop at musical time T in whole notes, e.g. <code>3/2</code>
--max-steps N	stop after N resolutions
--max-alternatives N	bound on maximal matchings per set (100000)
--unproductive N	bound on consecutive null-note steps (100000)

option	meaning (default)
--max-states N	bound on behavioural window graphs (200000)
--kmax K	interval-decoding digit bound per choice (64)
--trace FILE	write a replayable JSONL trace
--format table json	performance on stdout (table)
--out FILE	performance as JSON to FILE
--include-null	include null notes in the printed performance
--midi FILE, --bpm N	Standard MIDI File, at N quarter notes per minute (120)
--live PORT	live MIDI to the first port whose name contains PORT (live builds)
--quiet	print no performance

With none of `--notes`, `--until` or `--max-steps`, the player warns that an unbounded score will play forever. Written scores such as *Twinkle* stop by themselves when nothing is left to communicate. Voices do not: always bound them.

5.4 Chance sources

Every automatic source is a stream of digits, and every choice is made by exact interval decoding. No floating point enters a weight or a choice, so the same sources yield the same run on every platform.

spec	source
prng:SEED	SplitMix64 feeding xoshiro256**, fixed in the source so that it cannot drift with a dependency
spigot:C/B	digits of the fractional part of constant C in base B, with C one of pi, e, ln2, liouville, champernowne, thue-morse
spigot:C/B@Q	the same, starting at digit Q. Use this when the first note is written rather than drawn.
human	an interactive chooser on the terminal (§5.7)

A set with a single alternative, which includes every null note, consumes no digits.

5.5 Stream labels

Choices are routed to sources through *stream labels*. Independent voices should draw from independent streams: the note’s product-law theorem holds surely only then.

- `Voice(...)` #A labels every receipt in that application’s expansion A, and continuations inherit the label of the receipt that fired them. Labels are metadata: they do not change the digest.
- An unlabelled receipt takes its subject’s timbre as its label, or `loc:<digest>` when the timbre is open. So `--stream vibes=human` addresses every unlabelled receipt in timbre `vibes`.
- A contention set draws from the stream of its least receipt occurrence. At a chimera that is the first-installed timbre’s stream, so a chimera is normally given one explicit label.
- *Factor routing*. `--stream A.pitch=... --stream A.dur=...` resolves a voice’s pitch and duration by two independent draws (Lemma 6.4 of the note). This happens only when both are bound, the set is a star, and each top-level factor reads only one side. Otherwise the draw is joint. The trace records which happened (`"routing": "factored"` or `"joint"`).

5.6 Schedulers, modes and collection

Schedulers. When several contention sets are due at the same onset, the scheduler orders them: `canonical` is a fixed order, `random:SEED` a seeded shuffle, and `by-timbre:A,B` resolves timbre A's sets first. Independent voices give the same performance under every scheduler. For example, Twinkle's `--out` files under `by-timbre:bass,piano` and `random:9` are byte-identical.

Modes. `max` is the semantics. `one` fires a single candidate per set and exists only for diagnosis (test T3).

Collection. Voices leave behind record locations whose alternatives went unplayed. They are inert, but they sit in the soup. `--gc freshness` removes record-shaped locations that no longer have an available candidate. It never changes a performance, and it guards its own soundness: spawning at a collected location is the run-time error `freshness-violated`. Use it for long runs of voices. It collects nothing on instruments.

5.7 Human choice

`--chance human`, or `--stream LABEL=human` for one stream, stops at every set with more than one alternative. It lists the alternatives as timbre, pitch, duration and carried next datum (after `->`), with their normalised probabilities for guidance, and reads an index from standard input:

```
step 1 at onset 0 (stream marimba):
[0] marimba E4 q -> C4      (0.100)
[1] marimba E4 q -> D4      (0.100)
...
[9] marimba E4 e -> G4      (0.100)
choose 0..9>
```

Only alternatives of positive weight are offered, so every trace a human produces is admissible. The chooser is step-wise, not real time. Binding `human` to one label and PRNGs to the rest gives a mixed performance, with a person improvising against algorithmic players. If input ends while a choice is pending, the run stops with exit 3, after printing what was played.

5.8 Outputs

Performance. This is the multiset of timed notes of positive length, sorted by onset, timbre, pitch and duration, so that equal performances produce byte-identical files. Times and lengths are exact rationals in whole notes. The table form:

onset	timbre	pitch	dur	length
0	piano	C4	q	1/4
0	bass	C3	h	1/2
1/4	piano	C4	q	1/4

The JSON form (`--format json`, or `--out FILE`):

```
{ "score": "sha256:aa81...d0e8", "end": "5/4", "notes": [
  { "onset": "0", "timbre": "piano", "pitch": "A2", "dur": "h", "len": "1/2" }, ... ] }
```

Trace. JSON Lines, with format `flr3score-trace/2`. The first line is a header:

- the player version, the score path and digest;
- the alphabets, with MIDI numbers and lengths, and the timbres;

- every source and stream binding, the scheduler, mode, locality regime, collection mode, and kmax.

Then comes one line per resolution:

```
{ "step":1, "onset": "0", "loc": "31933326af6fe475", "set":2, "alts":2, "chosen":0,
  "routing": "joint", "digits": { "B":1 },
  "records": [ { "onset": "0", "loc": "31933326af6fe475", "recv": [ "bass", "C4" ],
    "send": [ "bass", "h" ], "payload": [ "8307dfa7ddb16d9", "bass", "E4" ] } ] }
```

Each record gives the receipt's and the message's subject decorations and the payload decoration, with the payload process given by its digest. Notes are not stored; they are computed from records.

MIDI. `--midi` writes a format-1 Standard MIDI File, with a tempo track and one track per rendered timbre. Each track carries a program change from the timbre's `gm(N)` and the channel from its `on C`. A note's track is chosen by its own timbre, so a chimera's notes are routed note by note to whichever timbre won. Rests and notes in the reserved timbres produce no events. The division is the least number of ticks per quarter that makes every onset and length integral, if that is at most 32767. Otherwise it is 960, with rounding and a warning. `--bpm` is a rendering parameter only.

5.9 Exit codes and stop reasons

code	meaning
0	success
1	usage error (unknown option, missing value, missing file)
2	parse or elaboration error (and malformed traces for render)
3	run-time error: unproductive, freshness-violated, alternative limit, state bound, I/O
4	replay mismatch: digest differs, a choice emits different notes, or choices are left over

On success the player reports on standard error why it stopped: `Quiescent` (nothing left to communicate), `Notes`, `Until` or `Steps`. Examples are `Quiescent` after 22 steps, 22 notes and `Notes` after 96 steps, 32 notes. The performance goes to standard output and diagnostics to standard error, so `--format json > perf.json` is safe.

6 Writing scores

6.1 File structure

A score file is UTF-8, with comments `//` and `/* */`. It contains, in any order, an optional `score NAME`, imports, the three alphabet declarations, `pitchset`, `def` and `factor` items, and exactly one `play` item giving the initial configuration.

```
score First
import std
pitches { r, C4, E4, G4, C5 }
durations { q = 1/4, h = 1/2 }
timbres { piano = gm(0) on 1 }

// a melody over a sustained two-note chord
play line(base "Mel", piano) [ C4 q, E4 q, G4 q, C5 q ]
  | line(base "Lo", piano) [ C4 h, C4 h ]
  | line(base "Hi", piano) [ G4 h, E4 h ]
```

Parallel lines sound together; that is harmony as independence at composition. The run:

onset	timbre	pitch	dur	length
0	piano	C4	q	1/4
0	piano	C4	h	1/2
0	piano	G4	h	1/2
1/4	piano	E4	q	1/4
1/2	piano	C4	h	1/2
1/2	piano	E4	h	1/2
1/2	piano	G4	q	1/4
3/4	piano	C5	q	1/4

flr3score: Quiescent after 8 steps, 8 notes

`import std` brings in the standard library. `import "lib.score"` reads a file relative to the importing score; an imported file may contain only `def`, `factor`, `pitchset` and `import`.

6.2 Alphabets

Pitches. `pitches { ... }` lists the pitch alphabet *in order*; that order defines `step(k)`. The rest `r` is required and is unordered. A pitch in scientific notation (C4, F#3, Bb2) gets its MIDI number automatically, with C4 = 60. Any other identifier needs one: `deg7` = 67. Enharmonics are distinct data. Two abbreviations are available:

- `scale KIND LO .. HI`, inclusive, where `KIND` is one of `major` (`ionian`), `minor` (`aeolian`), `dorian`, `pentatonic` (`major-pentatonic`), `minor-pentatonic`, `blues` or `chromatic`. A flat low pitch (Bb3) names the scale with flats;
- `piano88`, which gives A0 to C8.

`pitchset NAME = {...}` declares a named subset for use in definitions.

Durations. Each duration is `NAME = RATIONAL` in whole notes: `q` = 1/4, `t` = 1/6 for a triplet eighth, `dq` = 3/8 for a dotted quarter. The null duration `eps` = 0 is implicit and may not be redeclared. Pitch and duration names must be disjoint.

Timbres. `NAME = gm(PROGRAM) on CHANNEL`, with a General MIDI program from 0 to 127 and a channel from 1 to 16. The channel defaults to the timbre's position, modulo 16. `dead`, `ctl` and `_` are reserved.

6.3 Processes

surface	meaning
0	the inert process
<@P, tau, d>	a name; _ for an open timbre or datum. Compound quotes are parenthesised: <@(P Q), piano, q>
@P	the general name <@P, _, _>
x!(Q, tau, d)	a message whose payload is the name <@Q, tau, d>
x!(Q)	a message passing the general name @Q
for (y <- x where psi) { P }	a receipt; the clause is optional (weight 1)
for (y1 <- x1 & y2 <- x2 where psi) { P }	a join (chord receipt)
P Q	parallel composition
*x	drop: run the process quoted by x
at 3/4 { P }	an entrance: stamp P at 3/4 (top level of play only; the default is 0)
base "tag"	a closed process used as a location; distinct tags give inequivalent bases
keyloc(P, k), keycode(P, k, tau)	a key's location and its code location on keyboard base P

In binder position _ is an unused binder. In decoration position it is the wildcard.

6.4 Definitions

Definitions are macros, expanded at elaboration time:

```
def N(L: proc, tau: timbre, p: pitch, d: dur; K: proc) =
  for (_ <- <@L, tau, p>) { K } | <@L, tau, d>!(0)
def P1 = base "P1" // no
  parameters
def touch(P: proc, k: pitch, d: dur): name = <@keyloc(P, k), _, d> //
  returns a name
```

- **Sorts.** The parameter sorts are `proc`, `name`, `timbre`, `pitch`, `dur`, `clause`, `string`, `pitchset`, `lists [S]`, `tuples (S1, S2)`, and `cont`. A `cont` is a definition of arity one in a name, passed by name, with _ marking the hole: `Req(_, tau, BS)`.
- **No recursion.** The call graph must be acyclic. A recursive definition is rejected with a pointer to §5.2 of the note: unbounded behaviour comes from reflection, as in *Voice*.
- **Comprehensions.** `par v in durations+, t in pitches { ... }` expands to a parallel composition. The generators are `pitches`, `pitches-r`, `durations`, `durations+` (positive durations), declared pitch sets, literal sets, and lists, which may be deconstructed as `par (tau, psi) in voices`.
- **Lines.** `line(L, tau) [ C4 q, E4 h, ... ]` is a right-nested chain of written notes. `then K` replaces the final 0, for handing a phrase to a responder.
- **Synchronous output.** `a ; b ; 0` runs `b` when `a`'s note has ended. A step is one touch or a parenthesised parallel group. `;` binds more tightly than `|`, so `a ; 0 | c ; 0` is two players. A `;` inside a message payload is an error. Inside definition arguments `;` separates arguments, so parenthesise a sequence there.
- **Strings.** `++` concatenates strings, as in `base tag ++ "/S"`.

6.5 Clauses

A clause is a *graded* formula with values in the non-negative rationals. It is built from factors, which can be named with `factor NAME = ....`

graded form	value
$3/2, 0.3, 20$	a constant; decimals are exact (0.3 is $3/10$)
<code>[b]</code>	1 if the crisp formula <code>b</code> holds, else 0
<code>psi * psi, psi + psi</code>	product (short-circuits on 0), sum
<code>table KEY { k: w, ..., _: w }</code>	lookup by the listed key; <code>_</code> is the default and an absent default is 0. Keys are <code>dur</code> , <code>step</code> (ranges such as <code>-1..1</code> allowed, but they must not overlap), <code>(pitch, carry)</code> , <code>(prev, dur)</code> , or tuples of view functions
<code>machine pitch { C4 -> E4 @ 2, ... }</code>	a table keyed by <code>(pitch, carry)</code> : a weighted automaton on pitches
<code>machine dur { q -> e @ 1, ... }</code>	a table keyed by <code>(prev, dur)</code>
<code>machine dur uniform, machine pitch uniform</code>	the complete uniform machine
<code>machine pitch dual {...}</code>	the dual (rhythm-leads) reading
<code>machine pitch import "f.machine"</code>	a machine exported by the instrument
<code>sum x in S { psi }</code>	a sum expanded at elaboration

Crisp formulae have `true`, `false`, `!`, `&&`, `||`, the comprehensions `any x in S { b }` and `all x in S { b }`, and these atoms:

atom	holds when
<code>pitch(p), dur(v)</code>	the candidate's note has this pitch, this duration
<code>carry(e)</code>	the datum carried to the next step is <code>e</code> ; <code>carry(_)</code> when it is open
<code>step(k)</code>	carried and current pitch are ordered and differ by <code>k</code> positions in the pitch alphabet
<code>held(t), prev(u)</code>	the location's last record carries pitch <code>t</code> ; the previous duration was <code>u</code>
<code>back(j, e)</code>	the record <code>j</code> levels into the past carries <code>e</code>
<code>last[m1 .. mk], call[...]</code>	the last <code>k</code> notes match; the same on a handed-over call
<code>at(s), passes(s)</code>	the location's quote, or the passed process, satisfies the spatial pattern <code>s</code>
<code>pitch#i, dur#i, carry#i</code>	per-pattern atoms of a chord receipt
<code>leads(b)</code>	the configuration after firing the candidate satisfies <code>b</code> (§6.6)

Spatial patterns are `<n, tau, d>!(s, tau, d)` for a message, `for(<n, tau, d>) s` for a receipt, `0, true, s | s, !s, s && s` and `@s`. In patterns, `?` matches any component, *including* a wildcard, while `_` matches only a wildcard. A location is never a wildcard, so write `<?, ...>` rather than `<_, ...>`.

Encourage or enforce. Crisp factors enforce an idiom, and graded factors encourage one. Two examples from the conformance scores:

```
factor rule = [!(any t in pitches { back(0, t) && back(1, t) && carry(t)
    )}] // no triple repeat
factor boost = [back(1, C4) && back(0, D4) && carry(E4)] * 20 + 1
    // favour C-D-E
```


6.6 The checkable fragment

A clause is admitted if it is in one of two classes.

- **Local.** No behavioural formula appears. The clause's value depends only on the candidate.
- **Window-behavioural.** `leads(phi)` appears, where `phi` is built from `true`, `false`, `live`, `!`, `&&`, `||`, `<g> phi`, `<> phi`, `nu X. phi` and `X`. Every clause in the score must read the past to a bounded depth. The player then decides `phi` exactly on a finite graph of configurations, truncated to that depth.

check reports which regime a score uses. Graded behavioural modalities and graded fixed points are rejected with a reason. A bare modality must go under `leads(...)`. Exceeding `--max-states` is an error, never a truncation. The use for all this is viability, meaning enforcement that never paints a voice into a corner:

```
factor viable = [leads(nu X. <!(any t in pitches { back(0, t) && back(1,
    t) && carry(t) })> X)]
play Voice(piano, style * rhythm * rule * viable, C4, q, "A") #A
```

6.7 The standard library

definition	what it is
<code>N(L, tau, p, d; K), line</code>	a written note, and a written line
<code>Rec, Fan(x, tau, over),</code>	the record, the fan of all continuations (over a pitch set), and
<code>Seed</code>	the virtual first record
<code>Step, StepOver</code>	one step of a voice: every key offered, the clause keeps the live one
<code>Req, Refresh, Server,</code>	the reflective machinery: request, code refresh, server
<code>ServerTo</code>	
<code>Voice(tau, psi, p0, u0,</code>	an unbounded voice with clause <code>psi</code> , first pitch <code>p0</code> , previous duration <code>u0</code> . <code>VoiceOver</code> takes a pitch set
<code>tag)</code>	
<code>DualVoice</code>	rhythm leads: the duration is chosen a note early and carried
<code>Hocket(tauA, psiA, tauB,</code>	two players sharing one line, note for note, possibly on different instruments
<code>psiB, p0, u0, tag)</code>	
<code>Call, Resp, Responder,</code>	call and response. The call hands over its whole history;
<code>RespAny, ResponderAny</code>	
<code>Key, Installed, KeyRefresh</code>	...Any answers in whatever timbre the caller chose
<code>Keyboard(P, tau, psi,</code>	self-re-arming keys
<code>over)</code>	
<code>Chimera(P, k, voices),</code>	every key of a pitch set on base <code>P</code>
<code>ChimeraKeyboard(P, over,</code>	
<code>voices)</code>	one key location holding a key per (timbre, clause)
<code>touch(P, k, d)</code>	
	a touch name with the timbre left to the instrument; write
	<code>touch(P, A3, q)!(0)</code>

Read `crates/score-syntax/std.score` for the definitions themselves. It is short and fully commented. `flr3score expand` shows what any use of it elaborates to.

7 Worked examples

7.1 A voice with a style

A voice draws each note from its clause. Here the clause is mostly stepwise motion, a small rhythm machine, and no rests:

```
score Walk
import std
pitches { r, scale major C4 .. C5 }
durations { e = 1/8, q = 1/4, h = 1/2 }
timbres { piano = gm(0) on 1 }

factor steps = table step { -1..1: 4, -2: 1, 2: 1, _: 0 } // mostly
stepwise
factor rhythm = machine dur { q -> q @ 3, q -> e @ 1, e -> e @ 1, e -> q
@ 2,
q -> h @ 1, h -> q @ 1 }
factor norest = [!carry(r)]

play Voice(piano, steps * rhythm * norest, C4, q, "W") #W

flr3score play walk.score --notes 12 --stream W=prng:5 --gc freshness --
midi walk.mid
```

onset	timbre	pitch	dur	length
0	piano	C4	q	1/4
1/4	piano	C4	q	1/4
1/2	piano	D4	q	1/4
3/4	piano	E4	h	1/2
5/4	piano	D4	q	1/4
...				

flr3score: Notes after 36 steps, 12 notes

Each note takes three resolutions: one note and two null notes of the reflective server. Change the seed to get another performance of the same score. Change one factor to move the style to a new instrument.

7.2 The shipped examples

file	what it shows	try
twinkle.score	written lines and harmony by independence; golden score of T12	play ... --midi t.mid
country_voice.score	a product of experts, style × physicality × rhythm, against a walking bass	--notes 32 --stream A=prng:3 --stream B=prng:4 --gc freshness
hocket.score	two players sharing a line, marimba and vibes alternating	--notes 16
call_response.score	a flute calls and a cello answers a fifth up after reading the call	--notes 8
four_hands.score	two keyboards and two players with synchronous output; golden score of T13	--include-null shows the 16 null notes
chimera.score	detached eighths sound on vibes and sustained halves on strings; the instrument chooses its timbre	--midi c.mid

Call and response, for instance, plays:

0	flute	C4	q	1/4
1/4	flute	E4	q	1/4
1/2	flute	G4	q	1/4
3/4	cello	G4	h	1/2
5/4	cello	D5	h	1/2
7/4	cello	C5	h	1/2

7.3 Transcendental numbers as players

The instrument's embedding of π and e is itself a score. Route the voice's pitch factor to the hexadecimal digits of π and its duration factor to the base-5 digits of e :

```
f1r3score play crates/score-conformance/scores/t4_embedding.score --notes
8 \
--gc freshness --stream A.pitch=spigot:pi/16@1 --stream A.dur=spigot:e
/5
```

The pitches after the written D4 are E4, D#4, r, F#4, A#4, G#4, G#4. On the 16-state alphabet C4 ... D5, r, those are the hex digits 4, 3, F, 6, A, 8, 8 of π . @1 skips the first digit, because the first pitch is written, not drawn.

7.4 Importing a drawn machine

A weighted automaton drawn in the F1R3Skein instrument is exported with indexed states:

```
machine 3 start 1 { 0 -> 1 @ 1, 1 -> 1 @ 3, 1 -> 2 @ 2, 2 -> 1 @ 2, 2 ->
2 @ 1 }
```

```
f1r3score machine import dur.machine --kind dur --score examples/
country_voice.score --name groove
```

```
factor groove = machine dur { h -> q @ 1, q -> q @ 3, q -> e @ 2, e -> q @ 2, e
-> e @ 1 }
```

The states map onto the score's alphabet by position. A duration machine uses the positive durations in declaration order. A pitch machine uses the ordered pitches followed by the rest. The state count must equal the alphabet size.

8 Troubleshooting

error: 'D4' is not a declared pitch or duration

Every datum and timbre must be declared. Exit 2.

error: recursive definition: 'A' refers to 'A' ...

Use reflection (a Server with its code on a code channel), as Voice does.

error: 'dead' is the reserved dead timbre ...

Only base, keyloc and keycode may produce it.

error: a behavioural modality is a formula of the configuration: write it under 'leads(...)'

A bare <> in a clause.

error: a table key is listed twice

Overlapping ranges in a step table, such as -1..1 and -2..2. List the outer values separately.

error: expected a process, found `1`
 Only 0 is a process literal; use base "... " for a fresh location.

warning: a subject has datum `'\_` ... can never communicate
 A name with no polarity. The score still runs, but that receipt or message is dead.

warning: a top-level receipt and message share a location but their
 subjects never match
 Same polarity, or timbres that cannot meet (both open, or different).

Quiescent after 8 steps, 2 notes (with a larger --notes)
 The score deadlocked: every candidate has weight 0. A crisp rule has painted a voice into
 a corner. Add a viability factor (§6.6), or soften the rule into a graded factor. Compare
 viable_absent.score with viable.score in the conformance scores.

error: unproductive
 Too many consecutive null-note steps: a reflective loop that never plays. Exit 3.

the run never ends
 A voice is unbounded. Give --notes, --until or --max-steps.

memory grows over a very long run
 Terms are hash-consed and never freed; each note interns about 80 fan records. --gc
 freshness trims the soup but not the arena. Split very long performances.

replay-mismatch: step N: the replayed choice emitted different notes
 The trace was edited, or the score changed in a way that keeps the digest but not the
 choices. That should not happen, so report it with both files.

digest mismatch: the trace is of score ...
 Replay against the score the trace was made from.

9 Known limitations and discrepancies

The following were found while preparing this guide, or are recorded in DESIGN.md. None affects the correctness of offline runs.

1. **No CI workflow in the repository.** portability.rs and DESIGN.md refer to .github/workflows/ci.yml, which does not exist. As a result, the wasm32 build and the differential are not run automatically. A proposed workflow is in §4.
2. **wasm32 is unverified.** The core crates are I/O-free by test, but their wasm build has not been seen to succeed.
3. **Cargo.lock is not committed**, which weakens the reproducibility the design otherwise guarantees.
4. **Live MIDI timing differs from the specification.** The specification has a playback thread fed through a bounded channel, with the engine running ahead by a look-ahead and never waiting on the clock. The implementation instead sleeps inside the engine's callback until each note-on is due, so the engine waits on the clock. Live MIDI has not been tested on a device.
5. **--live requires PORT**, matched as a substring, whereas the specification makes it optional. The ports subcommand exists only in live builds and is not listed in the usage text.

6. **Lints print as error: warning:** ..., even though elaboration succeeds and the exit code is 0.
7. **A stale `replay:FILE` hint.** The error for an unknown chance source suggests `replay:FILE`, which is not implemented as a source. Replay is the `replay` subcommand.
8. **Single-alternative events are not checked on replay.** Their `chosen` field is not validated, since nothing was chosen. Tampering with a real choice is detected.
9. **Command-line forms differ from the specification.** The specification writes `replay piece.score run.jsonl and machine import FILE --pitches SCORE`. The binary takes `replay TRACE --score SCORE and machine import FILE --kind pitch|dur --score SCORE`. This guide follows the binary.
10. **Deadlock looks like a normal end.** It ends as `Quiescent` with exit 0. Compare the note count with `--notes` to detect it.
11. **Scope.** The following are outside this version:
 - graded behavioural modalities and graded fixed points;
 - atoms reading the note's timbre (reserved for orchestration);
 - a real-time human chooser;
 - tempo as a graded dimension.
12. **`spigot_stream` is not used.** The upstream crate in `F1R3Games` had defects outside base 10 (`DESIGN.md`), so `score-chance` computes its own digits.

A Repository layout

path	contents
<code>crates/score-core</code>	exact rationals, alphabets, decorations with wildcards, hash-consed terms in congruence normal form, substitution, matching, records and ν , bases and key locations, SHA-256 digests
<code>crates/score-logic</code>	spatial, crisp and graded formulae, tables, the checkable fragment, candidate views, evaluation
<code>crates/score-syntax</code>	lexer, parser, elaborator, printer, <code>std.score</code>
<code>crates/score-chance</code>	PRNG, spigot digits, exact interval decoding
<code>crates/score-engine</code>	soup, candidates, contention sets, maximal matchings, time, schedulers, collection, factor routing, the behavioural explorer
<code>crates/score-render</code>	playback, performance and trace JSON, trace parsing, MIDI writer, live MIDI
<code>crates/flr3score</code>	the command-line player
<code>crates/score-conformance</code>	tests T0–T14, the scores and golden MIDI files they use, <code>gen_scores.py</code> , <code>differential.py</code> , <code>run_differential.sh</code>
<code>examples/</code>	six example scores
<code>DESIGN.md</code>	decisions, deviations, conformance results, what is and is not verified
<code>docs/</code>	<i>Scores as Processes</i> (PDF)

The five core crates depend only on the `num-*` crates and perform no I/O. The licence is Apache 2.0.

B Conformance tests

test	claim checked
T0	inertness of unplayed alternatives
T1	a voice's law is the normalised product of its factors (20,000 notes)
T2	independent voices: interleaving invariance, product law
T3	a note cannot be fitted from both sides; <code>max</code> vs <code>one</code> mode
T4	the π/e embedding, by factor routing
T5	the dual (rhythm-leads) voice
T6	simultaneity by contention
T7	style $\times$ physicality
T8	motifs: forbid (crisp), encourage (graded)
T9	tables equal their formulae
T10, T11	reflective voice equals unrolled voice; inertness under reflection
T12	Twinkle: 22 notes ending at 4, under every scheduler
T13	4-hands piano: Example 9.5 exactly; a third player gives exactly two performances
T14	chimera: vibes share 0.9 / 0.5 / 0.1 by articulation; 1 when named

Measured values for each test are in `DESIGN.md`.